

Stage 6 实验报告

2022011223 熊泽恩

实验内容

Step 10

为了完成这次实验，我进行了以下工作：

词法分析

由于本 Step 中加入了全局变量，所以一个**程序**，除了可以由**函数**构成之外，还可以由**声明语句**构成。所以，我在 `ply_parser.py` 里修改了 `p_program` 规则，让 `program` 由一个或多个 `sentence` 构成，而 `sentence` 可以是 `declaration` 或者是 `function`。

语法分析

我修改了 AST 中 `Program` 的类型，让它的子结点既可以为 `Function` 类型，也可以是 `Declaration` 类型。

语义分析

在 `namer.py` 中，我检查了是否存在**全局变量重复定义**的情况。如果存在，则抛出异常。

生成 TAC

我新增了 `GlobalVar` 类，用来表示全局变量。

在 `tacgen.py` 的 `transform` 函数中，我新添了 `globalVars` 这一返回成员，包含所有的全局变量的信息（是否初始化、初始化为多少）。所有的全局变量位于作用域栈栈底的**全局作用域符号表**中，一直都存在，不会被弹出。

我在 `tacinstr.py` 中新增了三种指令：`GlobalAssign`、`Load` 和 `LoadSymbol`。

- `GlobalAssign` 的作用是给一个全局的地址赋值。
- `Load` 的作用是加载一个地址对应的值，类似于 C 语言中的 `*` 解引用。
- `LoadSymbol` 的作用是加载一个符号对应的地址，类似于 C 语言中的 `&` 取地址。

生成 RISC-V

我在 `riscasmemitter.py` 中添加了相关语句，将全局变量分为**已初始化的**和**未初始化的**，放到特定的数据段中。

- 已初始化的全局变量放到 `.data` 段中，并用 `.word` 来指定初值。
- 未初始化的全局变量放到 `.bss` 段中，并用 `.space` 来预留空间大小。

我在 `riscv.py` 中添加了 `GlobalAssign`、`Load` 和 `LoadSymbol` 三条对应的后端指令，它们分别被翻译为 `sw`、`lw` 和 `la`。

- `GlobalAssign`：用于存储全局变量的值，因为全局变量是以地址的形式被访问的，所以使用 `sw` 指令来存储数据。
- `Load`：用于从指定地址加载数据，可以使用 `lw` 指令来获取存储在内存中的值。

- `LoadSymbol`：用于加载一个符号对应的地址，可以使用 `la` (load address) 指令来获取符号的内存地址。

Step 11

为了完成这次实验，我进行了以下工作：

词法分析

由于 Step 11 中需要支持定义数组，所以我在 `lex.py` 中加入了 `t_LBracket = '['` 和 `t_RBracket = ']'`。

在 `ply_parser.py` 中，新加入了**定义数组**和**访问数组**的规则。

- 定义数组：定义数组由**数据类型**、**标识符**和**方括号内的维度**构成。
- 访问数组：访问数组由**标识符**和**方括号内的索引**构成。

而方括号的数量可以是一个或多个，所以也需要用规则进行约束。

语法分析

我在 `tree.py` 中加入了 `ArrDeclaration` 和 `ArrayAccess` 两种 AST 上的节点类型。

- `ArrDeclaration` 用于定义数组，在初始化时会**检查每一个维度的大小是否为正数**，否则抛出异常。
- `ArrayAccess` 用于访问数组，它由基址 `base` 和下标 `index` 构成。

语义分析

由于 Step 11 里引入了数组，现在我们的变量类型不只是 `int` 型了，还包括 `int` 型数组。因此，为了保证所有表达式中变量的类型均合法，需要进行类型检查。

一个数组如果是完整的，那么它可以是右值，也可以是左值；但如果它是不完整的，那么它既不是右值，也不是左值。

生成 TAC

我新建了 `ArrSymbol` 这个类，用于和 `VarSymbol` 相区分：前者是专门指代**数组类型**的符号。其余的工作就是修改 `tacgen.py`。

在 `visitIdentifier` 这个函数中需要注意，数组或者全局变量的符号查找得到的结果应该是一个地址，所以

- 如果是**数组类型或者是全局变量**，那么这个节点对应的属性是 `addr`，而非 `temp`。
- 如果是局部变量，那么才用 `temp`。

我添加了 `visitArrDeclaration` 这个函数，它

- 首先获取数组的符号
- 然后根据数组的类型大小，用 `visitAlloc` 分配相应的内存空间
- 最后将这个地址赋值给符号的 `addr` 属性。

我还添加了 `visitArrayAccess` 这个函数，它

- 首先访问数组的基址和索引，然后计算出数组元素的内存地址。
- 如果访问的是右值，并且访问的符号类型不是数组类型，则设置返回值。

生成 RISC-V

我在 `risc.py` 中，添加了 `Alloc` 这种指令，它对应着 `riscvasmemitter.py` 中的 `alloc` 函数。

这种指令主要用于**在栈上分配空间**，所以在函数内定义局部数组时会使用到它。

Step 12

为了完成这次实验，我进行了以下工作：

词法分析

数组传参

在定义函数的时候，如果参数为数组类型，那么它的**第一维可以为空**。我在 `ply_parser.py` 中添加了这一规则，最后得到的是一个 `list[int | None]` 类型的列表。如果为 `None`，则表示该维度为空。

数组初始化

数组初始化的值的格式为两个大括号内，有 0 个或多个以逗号分隔的整数。所以我先定义了 `p_integer_list` 这一个规则用来生成 0 个或多个以逗号分隔的整数，然后再添加 `p_array_declaration_init` 这条规则用于生成数组初始化语句。

语法分析

我在 AST 上增加了 `ArrParameter` 这个类型，用于表示**数组类型的函数参数**。它有一个 `list[int | None]` 类型的成员：`sizes`，用来表示数组参数的各个维度。只有第一维可以是 `None`，用来表示第一维为空的情况；且所有的维度大小均应是正数，否则抛出异常。

最后，为了方便，在检查后，我们将第一维为空的情况的维度大小，赋值为 0。

AST 上的 `ArrDeclaration` 类型也需要进行修改，因为需要完成数组初始化，所以添加了 `init_exprs` 这个成员变量。此时也需要进行检查：检查初始化的值的个数是否小于等于数组大小，否则抛出异常。

语义分析

我补充了 `typer.py`，用于判断每个操作符左右值的类型是否匹配。

但此时，由于**数组参数**的存在，数组作为左值/右值的规则发生了改变。一个数组如果是完整的，那么它可以是右值，也可以是左值；但如果它是不完整的，那么它可以是右值，例如进行参数传递，但它不能是左值。

在比较**定义时的数组参数**和**传入的数组参数**是否匹配时，需要忽略第一维，所以应该比较它们的 `base` 是否相等。

生成 TAC

数组传参

在 `tacgen.py` 中，我新添了 `visitArrParameter` 这个函数，它和已有的 `visitVarParameter` 是类似的。

数组初始化

我新增了 `GlobalArr` 类，用来表示全局数组。

在 `tacinstr.py` 中，我新增了 `Memset` 这种 TAC 指令，用于数组的初始化。

在 `tacgen.py` 中，我新增了 `visitInitArray` 函数，用于初始化数组。

- 首先，为每个初始化列表中的 `int` 型数分配一个临时变量；

- 然后，对数组进行全 0 赋值（使用 `Memset`）
- 最后，将初始化列表中的临时变量再赋给已初始化的数组地址。

这样，就能保证如果初始化时指定的元素个数比数组大小少，剩下的元素都会被初始化为 0。

在 `tacgen.py` 的 `transform` 函数中，我新添了 `globalArrs` 这一返回成员，包含所有的全局数组的信息（是否初始化、初始化为多少）。

生成 RISC-V

数组传参和普通的变量传参没有太大区别，重点还是在于怎么翻译数组初始化。

首先，我在 `riscvemitter` 中间添加了 `memset(addr, value, size)` 这个函数的定义，用于对于一段连续地址进行赋值。它接受起始地址、值和数组长度三个参数。代码逻辑大致如下：

```
1  memset_loop:
2      if (size == 0) goto memset_end
3      size -= 4
4      *addr = value
5      addr += 4
6      goto memset_loop
7  memset_end:
8      ret
```

然后，由于在 Step 12 中全局数组也可以被初始化，所以也需要将已初始化的和未初始化的全局数组分别放到 `.data` 和 `.bss` 段中。

Stage 6 思考题

Step 10

第 1 题

写出 `la v0, a` 这一 RISC-V 伪指令可能会被转换成哪些 RISC-V 指令的组合（说出两种可能即可）。

第一种

```
1  lui v0, %hi(a)
2  addi v0, v0, %lo(a)
```

先将 `a` 的高 20 位加载到 `v0` 中，再将 `a` 的低 12 位累加至 `v0` 中。

第二种

```
1  auipc v0, %pcrel_hi(a)
2  addi v0, v0, %pcrel_lo(a)
```

`%pcrel_hi(a)` 是 `a` 相对于当前 PC 的高 20 位，`%pcrel_lo(a)` 是 `a` 相对于当前 PC 的低 12 位。先将高位的相对值加上当前 PC 值（`auipc` 导致的），加载到 `v0` 中，再将低位的相对值累加至 `v0` 中。这样，合起来的相对值，再加上 PC 值（`auipc` 导致的），就是 `a` 的地址，因此能够实现 `la v0, a`。

Step 11

第 1 题

C 语言规范规定，允许局部变量是可变长度的数组 ([Variable Length Array](#), VLA)，在我们的实验中为了简化，选择不支持它。请你简要回答，如果我们决定支持一维的可变长度的数组（即允许类似 `int n = 5; int a[n];` 这种，但仍然不允许类似 `int n = ...; int m = ...; int a[n][m];` 这种），而且要求数组仍然保存在栈上（即不允许用堆上的动态内存申请，如 `malloc` 等来实现它），应该在现有的实现基础上做出哪些改动？

提示：不能再像现在这样，在进入函数时统一给局部变量分配内存，在离开函数时统一释放内存。

你可以认为可变长度的数组的长度不大于0是未定义行为，不需要处理。

首先，需要支持定义数组时方括号内为**表达式**而不仅仅是 `INT` 类型常量。

其次，由于 VLA 在运行时才能确定大小，所以不能在函数入口时统一分配内存，而是在计算出需要的空间大小后再分配。

- 在定义 VLA 时，根据计算出的长度动态调整 `sp`。
- 在离开作用域时，通过调整 `sp` 来释放栈空间。

Step 12

第 1 题

作为函数参数的数组类型第一维可以为空。事实上，在 C/C++ 中即使标明了第一维的大小，类型检查依然会当作第一维是空的情况处理。如何理解这一设计？

这是因为 C/C++ 中采用的通过指针传递数组参数。这样不需要传递整个数组，能减少内存使用。

此时，函数接收的是一个**指向数组首元素的指针**，所以第一维的大小并不重要。比如，`void func(int arr[] [5]);` 实际上等价于 `void func(int (*arr)[5]);`，在调用时，传的是一个**指向 `int[5]` 的指针**。